A cubic-communication variant of Multimmit

Small view-exit certificates and recovery from existing votes

Outcome. The construction below gives worst-case $O(Kn^2 + n^3)$ consensus communication for *each logical view*, hence $O(n^3)$ for $K = \Theta(n)$. It works for every configured fault bound $n \geq 5f+1$, including $f \ll n$. It changes the view-progression rules, retains full atomic votes and the original extraction rules, and supplies an explicit mechanism for recovering every referenced V-QC. The conservative timeout is 4Δ. A correct leader's block finalises within 5δ of the first correct entry into its view, and within 2δ of the leader's proposal transmission.

Roadmap. Small support certificates first restore sequential view progress without forwarding full V-QCs. A background rule then resolves each departed view into a genuine V-QC or a nullification. The remaining obstacle is recovery of an exact, possibly selectively disclosed, parent V-QC. Every correct recipient eventually knows all selected correct votes already, so only at most f records can be missing. We encode the repair information for those records, and spread it through the correct validators whose votes establish the proposal's validity. Sections 3–5 give the reconstruction and liveness proofs; Section 6 counts every new send.

Scope and status. This is a proposed replacement for the communication modification in the reviewed supplement, with a mathematical proof for the fixed-committee core. It is not a claim that the payment/epoch extension or the paper's separate lower bounds have been repaired. The quorum and extraction arguments of the core are reused under the explicitly proved witness property below. The companion manuscript version is `multimmit-31_10_26_1300.tex` [1]; this document replaces the earlier dissemination supplement in full. The construction has been checked against the counterexamples in the third correctness review; the coding component has also been tested independently. It is not a mechanised proof.

1 Model and the unchanged core interface

There are n validators, at most f Byzantine, and K transaction chains. The committee is fixed and $n \geq 5f + 1$. After global stabilisation, a message sent by a correct validator at time s reaches every intended correct recipient by $\max\{s, \mathrm{GST}\} + \delta$, with known $\Delta \geq \delta$. This is the delivery convention in the revised manuscript. Local enabled actions are exhausted within each timeslot. Cryptographic verification and local coding are treated as local computation, as are the paper's other certificate computations.

Hashes are collision resistant. Ordinary signatures, aggregate signatures and the existing $(2f + 1)$-threshold-signature scheme retain the paper's security assumptions. New threshold messages use a separate domain tag; no new threshold key or succinct-proof assumption is needed.

The following core objects and rules are retained.

- A *full vote* for a leader block ℓ contains the complete position and extension vectors, authenticated atomically by one ordinary signature. With fixed proposal and extension windows, its message body has size $O(K)$. A correct validator sends at most one such vote per view and never both a vote and a novote for that view.
- A *full V-QC* contains $n - f$ authenticated vote/novote records from distinct validators, including at least $2f+1$ full votes for its designated leader block. It has worst-case size $O(Kn)$. An *L-QC* contains $n - f$ full votes for one leader block.
- A *nullification* NC_u is the existing $(2f + 1)$-threshold signature on $(\mathsf{nullify}, u)$. An unvoted validator's timeout sends both a novote and a nullify share. A validator that has voted for

ℓ sends a nullify share only after obtaining $2f + 1$ messages from distinct validators, each a novote, nullify share, or full vote for a different leader block of that view.

- A directly valid proposal for view v references a valid full V-QC Q from a view $q < v$, proposes chains well formed relative to $\mathsf{Tips}(Q)$, and is accompanied locally by nullifications for every $q < u < v$. The leader-signature and no-known-equivocation checks remain.
- $\mathsf{Tips}(Q)$ is computed from the full votes in Q, and $\mathsf{Ord}(Q)$ recursively uses the exact parent QCs named by the leader blocks. The rules deriving finalised tips and emitted prefixes from full vote pools are unchanged. No small certificate below is an input to these extraction functions.

Retain the original signed messages, recovered QCs, manifests and recovery shards needed by later references, as in the core's retained-message model. An independent storage or pruning optimisation needs a separate argument.

Communication counts prescribed transmissions by correct validators. Arbitrary Byzantine flooding cannot be bounded. As in the manuscript, hash, signature and bounded metadata sizes are suppressed. Transaction bodies and their chain-layer dissemination remain outside this consensus-traffic bound. The $O(n^3)$ claim is total communication, not a per-validator bound.

2 Small certificates for sequential progress

Every full vote broadcast now includes the complete $O(K)$-size leader block ℓ and a threshold share on

$$(\mathsf{support}, v, H(\ell)).$$

The support share is sent *only together with that validator's actual full vote for the same block.* It is not sent merely because a certificate was received. The additional commitment in ℓ, defined in Section 3, is included in $H(\ell)$ and therefore in both the ordinary vote and this support share.

Definition 1 (Support certificate)**.** A support certificate $\mathsf{SC}_v(\ell)$ consists of the leader block ℓ for view v and a valid $(2f + 1)$-threshold signature on $(\mathsf{support}, v, H(\ell))$. Its size is $O(K)$.

The modified rules are as follows.

P1. Forward small exit evidence. Each correct validator forwards its first valid support certificate for each view once to all validators. It also forwards nullifications as in the original protocol. Support certificates for the same view need not designate the same block; uniqueness is not assumed. There is no all-to-all full-V-QC forwarding.

P2. Traverse views sequentially. A validator exits its current view v only on $\mathsf{SC}_v(\ell)$ or NC_v, and enters $v + 1$. Later-view evidence is cached. Several consecutive views may be processed within one timeslot, but no intermediate view is skipped. Remove the advancement-certificate and capped status/pull rules of the supplement.

P3. Vote before a support exit. On $\mathsf{SC}_v(\ell)$, an unvoted, non-nullified validator first casts its full vote for ℓ, using its own position and extension vectors, and broadcasts that vote, ℓ, and its support share. It then exits. The one-vote and vote/novote guards remain. A nullification exit need not cast a vote.

P4. Full V-QCs do not independently trigger exit or bypass voting. They remain valid anchors and extraction objects. Direct proposal-validity voting and support-certificate voting are the two voting routes.

P5. Continue old-view nullification. Retain each entered view's voted block and nullified flag after leaving it. Continue applying the original opposition-triggered nullify rule to these old views, sending at most one nullify share per view. Old timers need not continue.

P6. Use a 4Δ active-view timeout. Direct-validity voting and support-certificate vote-before-exit have priority over timeout at the same timeslot. The timeout sends a novote and nullify share only when the original unvoted/non-nullified guards hold.

P7. Leaders supply genuine anchors. A correct leader proposes once, as soon as it has a valid full V-QC anchor and the required gap nullifications. It chooses the greatest available anchor view below its current view, with a fixed tie-break, and broadcasts the full anchor with its proposal. Its one full-anchor broadcast is charged to this proposal view. It can compute the recovery manifest locally from the anchor.

The ordinary signed vote remains intact: the support share certifies a separate view-progression assertion. A support certificate may exist before any $n - f$-message V-QC can be assembled. The next leader therefore waits for a real anchor. The proof in Section 5 shows that this wait terminates without a large-object pull protocol.

Attaching ℓ to every full-vote broadcast is necessary. Otherwise, recipients might collect enough matching votes without obtaining their proposal body: a voter may already have used its single support-certificate forwarding allowance for a different block. This attachment costs only $O(K)$ per vote.

3 Exact-QC recovery from existing votes

The main observation is that all correct full votes and novotes are already broadcast. For any particular valid V-QC, every correct recipient eventually obtains every selected correct signer's record. Thus at most f selected records can remain missing. It is unnecessary to redisperse n records to repair those missing f.

3.1 Canonical representation and a precise manifest

Fix a canonical, injective QC representation. A full QC Q is identified by its designated leader block, its selected signer bitmap, the selected *complete unsigned message records* in signer-index order, and its aggregate signature. Define $H(Q)$ on that canonical representation. The current standard-vote/deviation compression may still be used on the wire, provided decompression yields this representation. Different lossless wire compressions of one QC then have one identifier.

Individual constituent signatures are not part of the recovery rows: the aggregate representation may not expose them. The aggregate signature is retained and verified against all reconstructed message records.

Write X_i for the unsigned record selected from signer i, with a zero row in an unselected slot. Records include their message type and all signed vote content; use an unambiguous fixed-width padding convention. Choose a finite field large enough for the codes below and represent each row by a vector in $\mathbb{F}^B$. Here B represents $O(K)$ message data, up to field-symbol padding. The selected-signers bitmap distinguishes absent slots from actual records.

The *manifest* $M(Q)$ consists of:

1. $H(Q)$, the designated leader block, the selected-signers bitmap, the aggregate signature, and the canonical format/length parameters;
2. the n indexed row hashes $H(X_1), \ldots, H(X_n)$;
3. the n indexed recovery-shard hashes defined below.

All hashes bind their context and index. The $O(K + n)$-size manifest uses full hash vectors, avoiding Merkle proofs within the cubic budget.

3.2 Two levels of erasure coding

Assume first that $f > 0$. Fix an $f \times n$ Cauchy matrix A over $\mathbb{F}$, so every square submatrix is invertible. For example, choose pairwise distinct field elements $a_1, \ldots, a_f, b_1, \ldots, b_n$ and set $A_{ji} = 1/(a_j - b_i)$. Compute the f parity rows

$$R_j = \sum_{i=1}^{n} A_{ji} X_i, \qquad 1 \le j \le f. \tag{1}$$

These f rows recover any f missing original rows when the other rows are known. The invertibility property follows from the Cauchy determinant formula; it is the standard Cauchy Reed–Solomon construction [2].

Concatenate the parity rows into R, of length fB field symbols. Pad and split R into $f+1$ equal vectors, then apply a fixed $(n, f+1)$ MDS erasure code, giving indexed shards $c_1(Q), \ldots, c_n(Q)$. Any $f+1$ distinct shards reconstruct R. For example, each coordinate of the input vectors can be treated as the coefficients of a degree-at-most-f polynomial, evaluated at n distinct field points; $f+1$ evaluations recover the coefficients by interpolation [3]. Each shard has length

$$\left\lceil \frac{fB}{f+1} \right\rceil = O(B) = O(K), \tag{2}$$

with the small symbol-padding term understood. This bound remains valid when f is much smaller than n. For $f = 0$, set every $c_i(Q)$ to the empty string and use its indexed hash in the manifest. Omit both coding steps, but still broadcast the manifest: every selected record is already broadcast by a correct signer. Genesis is globally known and requires no recovery.

3.3 Bind and transmit the recovery information

Replace a leader block's tuple by

$$\ell = (v, q, r, C), \qquad q = H(Q), \quad r = H(M(Q)).$$

Both the leader signature and $H(\ell)$ bind r. A *direct-valid vote* is a vote made after checking the full proposal-validity conditions, including possession of the exact full Q and recomputation of $M(Q)$, its row hashes, parities, and shard hashes. A correct direct-valid voter i broadcasts

$$(v, H(\ell), M(Q), i, c_i(Q)) \tag{3}$$

before or atomically with its ordinary vote and support share. The proposal body accompanies the vote. A recovery packet may be combined with that same broadcast. A validator sends at most one recovery packet in a view, for its one direct-valid vote; it does not send responses to arbitrary requests. A support-route vote without the checked full anchor sends no such packet. Previously initiated sends continue after view advancement.

Recipients check $H(M(Q)) = r$, the shard index and its committed hash, and retain only hash-matching candidate row records. They can use the correct voters' original broadcasts as these records. When enough matching shards and records arrive, they reconstruct Q, check $H(Q) = q$, and verify its full V-QC structure and aggregate signature. They also check the deterministic manifest on the reconstructed QC. A wrong Byzantine record becomes an erasure; it is never used as an unchecked equation input.

Lemma 2 (Direct-valid witnesses)**.** *If ℓ has a support certificate, or is designated by a valid full V-QC or L-QC, then at least $f+1$ correct validators have cast direct-valid votes for ℓ.*

Proof. Work in any finite execution prefix in which the hypothesised certificate already exists. If a correct support-route vote for ℓ exists in that prefix, consider the earliest such vote in causal order. Its support certificate requires $2f + 1$ earlier support shares, including at least $f + 1$ from correct validators. Each of those shares accompanies an actual earlier full vote for ℓ. By minimality, those correct votes are direct-valid votes. If there is no correct support-route vote for ℓ, then the at least $f+1$ correct votes guaranteed by the hypothesised certificate are themselves direct-valid. This argument uses the rule that observing a support certificate alone does not cause a fresh support share without a full vote. □

Lemma 3 (Exact recovery)**.** *For every proposal ℓ covered by Lemma 2, every correct validator eventually obtains its exact referenced full V-QC Q. If a valid support certificate, full V-QC or L-QC for ℓ is held by a correct validator at time s, recovery completes everywhere by* $\max\{s, \mathrm{GST}\}+\delta$.

Proof. Apply Lemma 2 to the execution prefix ending at s. Its $f+1$ correct direct-valid witnesses hold the same exact Q and recompute the same deterministic manifest. Their distinct indexed recovery shards were broadcast by s. All correct recipients therefore obtain $f + 1$ authenticated shards and recover R by the stated deadline.

Every selected correct row of Q was also broadcast by its signer before Q could be checked by those witnesses. These rows arrive by the same deadline. Unselected zero rows are known from the bitmap. The remaining unknown selected rows form a set U of size $u \le f$. Subtract known contributions from the first u parity equations to obtain

$$R_j - \sum_{i \notin U} A_{ji} X_i = \sum_{i \in U} A_{ji} X_i, \qquad 1 \le j \le u.$$

The coefficient matrix is an invertible Cauchy submatrix. Solve coordinatewise to recover all missing rows, then reconstruct the exact canonical QC using its aggregate signature and header from the manifest. Hash and full certificate checks authenticate the result. When $f = 0$, all rows arrive directly and the same conclusion holds without parity. □

Applying this argument to the finite descending chain of referenced views recovers every QC required to compute Ord. Different Byzantine vote variants cause no ambiguity: the manifest selects the exact row hashes used by the named QC. Multiple proposals in one faulty-leader view do not multiply any correct sender's allowance; each correct validator votes and sends its recovery contribution at most once in that view.

4 Safety argument and proof changes

The extraction certificates and their complete atomic vote contents are unchanged. The new manifest hash is merely an additional field of the leader-block identifier. All position and extension computations still use the same chain proposal C and the voter's own DA history.

The essential proof adaptation is Lemma 2. In the original leader-chain and availability arguments, a V-QC-triggered vote is traced back to earlier correct voters that checked the proposal. The new support route has the same required $f + 1$ direct-valid witnesses. Consequently, every certified proposal inherits an actually checked full parent QC, well-formed chain proposal, and gap nullifications. A support certificate never supplies positions or substitutes for a full QC in a quorum count.

One-vote-per-view and vote/novote exclusivity follow from the retained historical flags. If a block receives an L-QC, a support certificate for a different block of that view is impossible: at

most f correct validators can lie outside the L-QC's correct supporters, and at most f faulty validators can add conflicting shares, fewer than $2f+1$. The original no-nullification argument also remains valid with background processing: if a block obtains the original large vote quorum, consider the first correct supporter that might later nullify. Its $2f+1$ opposing messages cannot exist, using the same correct-signer count as in the core proof. This argument concerns all messages ever sent for that view, so leaving the view does not change it.

The leader-chain argument can therefore be rerun with the witness lemma. The safe-extension and quorum-intersection calculations continue to use the original full votes and full QCs; their thresholds do not change. The emission-prefix argument uses the current pool's settledness, as identified in the third review: reaching the relevant second-pass slot without halting gives

$$\beta_j + (n - |P|) \leq f,$$

and completing the first pass gives $k_j^* = m_j$. These establish settledness relative to P, allowing Safe extension (b) to supply the required tip equality. This is the independent local correction to retain when invoking consistency.

Thus the operational core's consistency and inclusion arguments transfer with this stated change to the causal voting lemma and the existing local pool correction. The separate universal lower-bound proofs are not used to establish the communication construction.

5 View progress, usable anchors and finalisation

Lemma 4 (Sequential synchronisation)**.** *If the first correct entry into view v occurs at time t, every correct validator enters a view at least v by* $\max\{t, \mathrm{GST}\} + \delta$.

Proof. The entering validator has processed every view $u < v$. For each it has forwarded a support certificate or nullification by time t. The forwarded support certificate need not be the same variant it eventually used: either is adequate exit evidence. By the stated deadline every correct validator has exit evidence for every $u < v$, and processes the views sequentially within the timeslot, casting any prescribed votes before advancing. □

Lemma 5 (Resolution of a departed view)**.** *Let s be the first correct entry into view $u+1$, and put $S = \max\{s, \mathrm{GST}\}$. By $S + 3\delta$, every correct validator has either a valid full V-QC for u or a nullification for u.*

Proof. By Lemma 4, every correct validator has passed through u by $S + \delta$. If any correct validator holds a nullification for u by $S + 2\delta$, forwarding puts it everywhere by $S + 3\delta$.

Otherwise no exit by $S + \delta$ can have used a nullification. Each correct validator exited on a support certificate. It had already voted, cast its prescribed vote before exit, or had previously sent a timeout novote. A validator that nullified without voting could only have done so by that timeout rule. Hence every correct validator broadcast a full vote or novote for u by $S + \delta$. Everyone obtains all these messages, with the full proposal bodies attached to votes, by $S + 2\delta$.

If one proposal has at least $2f+1$ correct votes, choose those and enough other correct messages to obtain a valid $n - f$-message V-QC. Everyone can assemble one locally. Otherwise each proposal has at most $2f$ correct votes. Every correct voter then has at least

$$(n - f) - 2f = n - 3f \geq 2f + 1$$

correct messages not supporting its chosen block. The retained background rule makes it send its nullify share, if it has not already done so. Novoters already sent theirs. These shares produce a nullification everywhere by $S + 3\delta$. □

The deadline split in this proof matters. A nullification exit can occur without a vote or novote. The proof only counts all correct view messages after excluding that case; it does not silently assume that every departure produces a vote.

Lemma 6 (A correct leader obtains an anchor)**.** *If the first correct entry into view v is at $t \geq$ GST, a correct leader can issue a valid proposal by $t + 3\delta$.*

Proof. Every first correct exit from an earlier view occurs by t. Apply Lemma 5 simultaneously to every such view. By $t + 3\delta$, the leader has a full V-QC or nullification for each $u < v$. Select the greatest view $q < v$ for which it holds a full V-QC, using genesis if necessary. Every $q < u < v$ must then have a nullification. Thus a valid anchor and all required gap evidence are available. If it could propose earlier, rule P7 already instructed it to do so. If it has not yet proposed, it is still in the view: no support certificate can exist without earlier correct votes for its signed proposal, and no nullification can precede the 4Δ timeout without a first correct nullifier or a competing correct vote. Thus P7 can execute. Coding the manifest introduces local computation but no additional communication prerequisite. □

Theorem 7 (Correct-leader finalisation)**.** *Let the first correct entry into a correct-leader view v occur at $t \geq$ GST, and let τ be the leader's proposal time. Then*

$$t \leq \tau \leq t + 3\delta,$$

all correct validators vote by $\tau + \delta \leq t + 4\delta$, and all obtain an L-QC and finalise the leader block by $\tau + 2\delta \leq t + 5\delta$.

Proof. The leader signs only one block for the view. No correct validator can vote for a different block: a first such vote is neither directly valid nor support-triggered, since the latter would require earlier correct votes for that same different block. No correct timeout occurs before $t + 4\Delta \geq t + 4\delta$. No earlier correct nullification can occur either: a first such event would need more opposing messages than the f faulty validators can supply, since correct competing votes, timeout novotes and earlier correct nullifications are absent.

By $\tau + \delta$, all correct validators have reached at least v by Lemma 4. They also have the proposal, its full anchor and all gap nullifications: the leader has broadcast the anchor and proposal, and forwarded the nullifications by τ. The deterministic manifest check is local. Validators still in v vote, with priority over any same-timeslot timeout. Any validator already beyond v must have exited on this proposal's support certificate and voted first. Hence all correct votes are sent by $\tau + \delta$, and all arrive by $\tau + 2\delta$, giving the required $n - f$-vote pools.

The current anchor is supplied in full. All of its proper reference-chain ancestors are also recoverable by $\tau + \delta$: their required correct recovery packets and selected correct records were broadcast before the top anchor could exist, by the causal proof of Lemma 3. Thus missing ordering metadata does not postpone the conclusion. Delivery of transaction bodies retains the core's separate data-retrieval convention. □

Lemma 8 (Progress through any view)**.** *If the first correct entry into view v occurs at $t \geq$ GST, all correct validators leave it by $t + 4\Delta + 3\delta$.*

Proof. If a correct validator exits by $t + 4\Delta + 2\delta$, synchronisation gives the result. Otherwise every correct validator remains in the view, having entered by $t + \delta$. Each sends its full vote or timeout novote by $t + 4\Delta + \delta$, so everyone has all correct messages by $t + 4\Delta + 2\delta$. If one proposal has $2f + 1$ correct votes, their support shares produce an exit certificate. Otherwise the opposing-message argument in Lemma 5 produces a nullification within one more delay. □

The same argument with $\max\{t, \mathrm{GST}\}$ starts from views whose first entry predates GST. Induction gives eventual traversal of every view. Round-robin leaders therefore yield recurring correct-leader finalisations. Together with exact reference recovery and the unchanged block-inclusion argument, this supplies the core's eventual transaction liveness.

6 Worst-case communication for each view

All limits below are permanent per-view limits; they do not reset when a request arrives or another validator advances. A send relating to an old view is charged to the view named by its action. A recovery contribution and the leader's full-anchor broadcast are charged to the view of the proposal that references that anchor.

Transmission class	Copies per view	Size per copy
Full votes, with proposal bodies and support shares	$O(n^2)$	$O(K)$
Novotes and nullify shares, including late closure	$O(n^2)$	$O(1)$
First support-certificate forwarding	$O(n^2)$	$O(K)$
Nullification forwarding	$O(n^2)$	$O(1)$
Correct leader's one full-anchor proposal	$O(n)$	$O(Kn)$
One recovery contribution per direct-valid voter	$O(n^2)$	$O(K+n)$

Theorem 9 (Per-view bound)**.** *For every execution and logical view, correct validators send $O(Kn^2+n^3)$ consensus data in $O(n^2)$ messages. In particular, for $K = \Theta(n)$, the total is $O(n^3)$ per view in the worst case, including before GST. The bound includes recovery of named ancestor QCs.*

Proof. Each correct validator sends at most one ordinary vote or novote, at most one nullify share, at most one recovery contribution, and at most one broadcast of each small certificate type per view. There is only one correct leader for a given view, and it sends at most one full anchor to each recipient. Sum the table. A Byzantine leader may equivocate, but cannot cause a correct validator to exceed its one-vote or one-recovery-contribution limit. A private Byzantine QC does not trigger full-QC forwarding. Recovery causes no additional request-response traffic. □

This is an independent cap for each logical view, not an amortised argument that pays for an expensive view with cheap later ones. Sequential catch-up may process a backlog of many old views in one physical interval; the theorem does not bound such a physical interval by the cost of one view. Through V logical views the total is $O(V(Kn^2+n^3))$.

For $K \ll n$, the present construction should be advertised as $O(Kn^2+n^3)$, not as $O(Kn^2)$. The indexed hash manifests contribute the n^3 term. Restoring security-parameter and field-symbol sizes adds their conventional factors; field-padding terms of $O(n^2 \log n)$ are absorbed by $O(n^3)$ under the paper's fixed cryptographic-size convention. A nonleader's prescribed outgoing traffic is $O((K+n)n)$; a correct leader additionally sends $O(Kn^2)$ for its full-anchor proposal.

7 Transferred results

The construction is designed so that the manuscript's results transfer with one substitution. The following proposition makes the transfer explicit, so that this document, together with the manuscript's proofs, is checkable line by line.

Proposition 10 (Carried results)**.** *Under the construction of Sections 2–3, the manuscript's [Designation], [Safe extension], [Availability], [Block inclusion], [Consistency] and [Emission is a prefix] hold verbatim, and [Liveness] holds with the constants of Section 5.*

Proof. Three kinds of dependence on the dissemination rules occur in the manuscript's proofs. First, pure counting over signed objects: full votes, V-QCs, L-QCs, pools and nullifications are byte-for-byte the manuscript's objects, and no support certificate enters any extraction, quorum or threshold count (rule P4), so [Designation], [Safe extension] and every quorum-intersection step are untouched. Second, causal valid-proposal inferences: where a proof traces a certificate back to a correct voter that checked the proposal in full — the anchor and base availability steps of [Availability], and the frontier-endorsement step of [Block inclusion] — Lemma 2 supplies at least $f + 1$ correct direct-valid voters behind every support certificate, V-QC and L-QC, each of which checked exactly the original validity conditions, full-anchor possession included; substituting these witnesses for the single checked voter the original arguments extract, the proofs go through unchanged. Third, exact references: where [Consistency]'s recursive Ord computation needs the exact parent QCs named on the reference chain, Lemma 3 delivers them to every correct processor, and [Emission is a prefix] uses the pool-settledness inference in the manuscript's corrected form, reaching a second-pass slot without halting giving the pool continuation condition, the completed first pass giving $k_j^* = m_j$, hence settledness in P and Safe extension (b). For [Liveness]: Lemma 8 and induction traverse every view, round-robin leadership supplies recurring correct-leader views, Theorem 7 finalises each within 5δ of its first correct entry after GST, and [Block inclusion]'s hypotheses are met as above, so the manuscript's transaction-liveness argument runs with these lemmas in place of the replaced synchronisation, exit and correct-leader lemmas. □

8 Integration, tradeoffs and validation

The replacement should be specified as one coherent variant. In particular, do not retain the supplement's skipping rule or its capped QC retrieval alongside these rules. Also remove the old full-V-QC bypass-vote and exit routes from the replacement pseudocode; their purpose is served by the new support certificate. Full V-QCs continue to determine ordering.

The proof changes are concentrated in the following places:

- extend the leader-block identifier and signature to bind the recovery manifest, and define canonical QC hashing;
- add support shares, the compact certificate, sequential vote-before-exit, historical nullification state, and the 4Δ timeout;
- replace the causal valid-proposal inference in the leader-chain and availability/inclusion proofs by Lemma 2;
- replace the view-synchronisation, view-exit and correct-leader timing lemmas with Section 5;
- replace the unspecified named-QC retrieval convention with Lemma 3, and retain the pool-settledness proof correction.

The main cost is additional local coding and hashing, plus the larger conservative timeout. There is no mandatory extra communication round between a correct leader's proposal and finalisation. An anchor may require up to the proved 3δ preparation interval after first view entry. The note does not claim the original 2Δ timeout or original 3δ entry-to-finalisation bound for this variant. Simple matrix implementations of the coding can be computationally expensive; the communication theorem does not establish an implementation throughput figure.

Checks performed. The view-progress argument was checked against the earlier alternating skipped-voter execution, conflicting support certificates, late nullifications, an old view with missing messages after a nullification exit, and arbitrary $n \geq 5f + 1$. Exact recovery was checked for selectively disclosed Byzantine records, different Byzantine variants, aggregate-only QC signatures, and $f \ll n$.

An independent finite-field implementation passed 11,930 reconstruction cases: exhaustive missing-row and recovery-shard subsets for $(n, f) = (6, 1)$ and $(11, 2)$; deterministic samples for $(16, 3)$, $(50, 9)$, and $(37, 1)$; and $f = 0$ cases. It also rejected 247 tampered row/shard inputs and passed five combined filtered-recovery cases. These are coding-component checks, not distributed-protocol model checking. The proof above supplies the protocol argument.

References

[1] Anonymous author(s). Multimmit: Faster finality for ordering and payments. Under submission, 2026.

[2] James S. Plank. *Optimizing Cauchy Reed-Solomon Codes for Fault-Tolerant Storage Applications*. Technical Report CS-05-569, University of Tennessee, 2005, Section 2. `https://web.eecs.utk.edu/~jplank/plank/papers/CS-05-569.pdf`.

[3] Anup Rao. *Lecture 4: Reed-Solomon Codes*. University of Washington, 2019, pp. 2–3. `https://homes.cs.washington.edu/~anuprao/pubs/codingtheory/lecture4.pdf`.